the returned set of elements. Notice that keys are preserved from the first array. `'Golf'` occurs twice in `$car1`, with the (automatically generated) keys of `0` and `2`, those keys are retained in the results array that is returned to `var_dump()`.

Also worth bearing in mind: `array_diff()` will sort arrays before comparison, and type coercion is the norm:

```
var_dump(array_diff([1, 2, 3], [3, 2, 1]));
var_dump(array_diff(
    ['make' => 'VW', 'model' => 'Golf'],
    ['model' => 'Golf', 'make' => 'VW'],
));
var_dump(array_diff(['1', '2', '3'], [1, 2, 3])); // type juggling will
                                                   happen here

Output:
array(0) {
}
array(0) {
}
array(0) {
}
```

Finding Common Elements

The corollary to computing the difference between sets is computing the intersect. This time we're finding the elements of the first array that also occur in subsequent arrays. Just as before, keys will be preserved from `$car1` and multiple occurrences of an element in the first array will mean multiple occurrences in the results array. This time elements are added to the results if they can be found in all comparison arrays.

Let's swap `array_diff()` for `array_intersect()` using the same data in the example above:

```
print_r(array_intersect($car1, $car2));

Output:
Array
(
```